

UNIT-3

OBJECT ORIENTED PROGRAMMING CONCEPTS

3.1 CLASS AND OBJECT

What is a class in Java?

A class is a group of objects which have common properties.

It is a template or blueprint from which objects are created.

It is a logical entity. It can't be physical.

A class in Java can contain:

- Fields / Variables
- Methods
- Constructors
- Blocks
- Nested class and interface

Syntax to declare a class:

```
class class_name{  
    // field;  
    // method;  
}
```

What is an object in java?

An entity that has state and behaviour is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

An object is an instance of a class.

An object has two characteristics:

- **State:** represents the data (value) of an object.
- **Behaviour:** represents the behaviour (functionality) of an object such as deposit, withdraw, etc.

Example: Pen is an object. Its name is Reynolds; colour is white, known as its state. It is used to write, so writing is its behaviour.

Object Definitions:

- An object is *a real-world entity*.
- An object is *a runtime entity*.

- The object is *an entity which has state and behaviour*.
- The object is *an instance of a class*.

Example of Class and Object:

```
class Student {
    int id;

    String name;

    public static void main(String args[]){
        //Creating an object or instance

        Student s1=new Student ();//creating an object of Student

        //Printing values of the object

        System.out.println(s1.id);//accessing member through reference variable

        System.out.println(s1.name);
    }
}
```

Output:

```
0
null
```

3.2 CONSTRUCTOR AND TYPES OF CONSTRUCTORS

In Java, a constructor is a special method that is used to initialize objects of a class. It is called automatically when an object is created using the **new** keyword. Constructors have the same name as the class and do not have a return type, not even **void**.

3.2.1 TYPES OF CONSTRUCTORS

1.Default Constructor:

A default constructor is provided by Java if you do not explicitly define any constructor in your class. It has no parameters and does not perform any initialization. Its purpose is to initialize the object with default values (e.g., setting numeric properties to 0 and reference properties to **null**).

EXAMPLE:

```
class Bike1{

    //creating a default constructor

    Bike1(){System.out.println("Bike is created");}
```

```
//main method
public static void main(String args[]){
    //calling a default constructor
    Bike1 b=new Bike1();
}
}
```

Output :

Bike is created

2. Parameterized Constructor:

A parameterized constructor is a constructor that takes one or more parameters. It allows you to initialize the object with specific values at the time of creation.

EXAMPLE:

```
class Person(var name: String, var age: Int) {

    // Parameterized constructor
    init {
        println("Name: $name, Age: $age")
    }

    // Method to display person details
    fun displayDetails() {
        println("Name: $name, Age: $age")
    }
}

fun main() {
    // Creating a Person object with parameterized constructor
    val person1 = Person("John", 30)
    // Displaying details of the object
    println("Person details:")
    person1.displayDetails()
}
```

3.Copy Constructor:

A copy constructor is a constructor that creates a new object by copying the state of another object of the same class. It is useful when you want to create a new object with the same values as an existing object.

EXAMPLE:

```
class Person(var name: String, var age: Int) {  
    // Primary constructor  
    constructor(person: Person) : this(person.name, person.age) {  
        // Copy constructor implementation  
        println("Copy constructor called")  
    }  
    // Method to display person details  
    fun displayDetails() {  
        println("Name: $name, Age: $age")  
    }  
}  
fun main()  
{  
    // Creating a Person object  
    val person1 = Person("John", 30)  
  
    // Displaying details of the original object  
    println("Original object:")  
    person1.displayDetails()  
  
    // Creating a new object using the copy constructor  
    val person2 = Person(person1)  
  
    // Displaying details of the copied object  
    println("\nCopied object:")  
    person2.displayDetails()  
}
```

3.3 METHOD AND METHOD OVERLOADING

Method is a collection of statements that are grouped together to perform a specific task. Methods are used to encapsulate code and provide reusability and modularity in your programs. They are defined inside classes and can be called to perform the actions they define.

Syntax:

```
modifier returnType methodName(parameterList) {
    // Method body
}
```

Example:

```
public class Calculator{
    //method of addition
    public int add(int num1, int num2) {
        return num1 + num2;
    }
    public static void main(String args[]){
        //Creating an object of Calculator class
        Calculator obj = new Calculator();
        //Calling an method
        System.out.println(obj.add(1,2));
    }
}
```

Output :

3

Types of Method :

1. no return no parameter :

Example :

```
void display(){
    System.out.println("Hello world");
}
```

2. no return with parameter :

Example :

```
void display(String msg){
    System.out.println("Your message is : "+msg); }
```

3. with return no parameter :

Example :

```
int number;

int returnNumber(){
    number = 5;
    return number; }
```

4. with return with parameter :

Example :

```
int add(int num1, int num2) {
    return num1 + num2; }
```

5. Recursive function :

A method in java that calls itself is called recursive method.

It makes the code compact but complex to understand.

Syntax :

```
returntype methodname(){
//code to be executed
methodname();//calling same method
}
```

Example :

```
public class RecursionExample3 {
    int factorial(int n){
        if (n == 1)
            return 1;
        else
            return(n * factorial(n-1));    }

    public static void main(String[] args) {
        RecursionExample3 obj = new RecursionExample3();
        System.out.println("Factorial of 5 is: "+ obj.factorial(5));
    }
}
```

Output :

Factorial of 5 is: 120

3.3.1 METHOD OVERLOADING

Method overloading is a feature in Java that allows you to define multiple methods with the same name but different parameters within the same class. The methods must have different parameter types, different numbers of parameters, or both.

USES OF METHOD OVERLOADING:

Method overloading when a couple of methods are needed with conceptually similar functionality with different parameters.

EXAMPLE:

```
public class Calculator {  
    // Method to add two integers  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method to add three integers  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method to add two doubles  
    public double add(double a, double b) {  
        return a + b;  
    }  
  
    // Method to add three doubles  
    public double add(double a, double b, double c) {  
        return a + b + c;  
    }  
  
    // Method to concatenate two strings  
    public String add(String a, String b) {  
        return a + b;  
    }  
}
```

```
// Method to concatenate three strings
public String add(String a, String b, String c) {
    return a + b + c;
}

public static void main(String[] args) {
    Calculator calculator = new Calculator();

    // Calling the overloaded methods
    int sum1 = calculator.add(5, 3);
    int sum2 = calculator.add(2, 3, 4);
    double result1 = calculator.add(2.5, 3.5);
    double result2 = calculator.add(1.5, 2.5, 3.5);
    String concat1 = calculator.add("Hello", "World");
    String concat2 = calculator.add("Java", " is", " awesome");

    // Outputting the results
    System.out.println("Sum of two integers: " + sum1);
    System.out.println("Sum of three integers: " + sum2);
    System.out.println("Sum of two doubles: " + result1);
    System.out.println("Sum of three doubles: " + result2);
    System.out.println("Concatenation of two strings: " + concat1);
    System.out.println("Concatenation of three strings: " + concat2);
}
}
```

3.4 THIS KEYWORD

‘**this**’ keyword is a reference to the current instance of a class. It is used within an instance method or constructor to refer to the object on which the method or constructor is being invoked.

Here are a few common uses of the **this** keyword:

1.Accessing Instance Variables:

You can use this to refer to the current object's instance variables when there is a naming conflict between instance variables and method parameters or local variables.

EXAMPLE:

```
public class MyClass {  
    // Instance variable  
    private int value;  
  
    // Constructor  
    public MyClass(int value) {  
        this.value = value; // Assigning value to the instance variable  
    }  
  
    // Method to display value  
    public void displayValue() {  
  
        // Accessing instance variable directly  
        System.out.println("Value: " + value);  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        // Creating an instance of MyClass  
        MyClass obj = new MyClass(10);  
        // Calling the method to display value  
        obj.displayValue();  
    }  
}
```

2.Invoking Another Constructor:

In a constructor, you can use this to invoke another constructor of the same class. It is useful when you want to reuse the initialization logic of one constructor in another constructor.

EXAMPLE:

```
public class MyClass {  
    private int value;  
  
    // Primary constructor  
    public MyClass(int value) {  
        this.value = value;  
        System.out.println("Primary constructor invoked with value: " + value);  
    }  
  
    // Secondary constructor invoking primary constructor  
    public MyClass(int value, String name) {  
        this(value); // Invoking the primary constructor  
        System.out.println("Secondary constructor invoked with value: " + value + " and name: " + name);  
    }  
  
    // Method to display value  
    public void displayValue() {  
        System.out.println("Value: " + value);  
    }  
  
    // Main method  
    public static void main(String[] args) {  
        // Creating an instance using primary constructor  
        MyClass obj1 = new MyClass(10);  
        obj1.displayValue();  
  
        // Creating an instance using secondary constructor  
        MyClass obj2 = new MyClass(20, "John");  
        obj2.displayValue();  
    }  
}
```

3.Returning The Current Instance:

You can use this to return the current instance of the object from a method. It is commonly used for method chaining or fluent interfaces.

EXAMPLE:

```
public class MyClass {  
    private int value;  
    // Constructor  
    public MyClass(int value) {  
        this.value = value;  
    }  
    // Method to increment value and return the current instance  
    public MyClass increment() {  
        value++;  
        return this; // Returning the current instance  
    }  
    // Method to display value  
    public void displayValue() {  
        System.out.println("Value: " + value);  
    }  
  
    // Main method  
    public static void main(String[] args) {  
  
        // Creating an instance of MyClass  
        MyClass obj = new MyClass(10);  
  
        // Invoking method chaining and displaying the value  
        obj.increment().increment().increment().displayValue();  
    }  
}
```

4. this can be used to invoke current class method (implicitly).

5. this can be passed as an argument in the method call.

6. this can be passed as argument in the constructor call.

3.5 STATIC KEYWORD

The **static keyword** is used to declare members (variables and methods) that belong to the class itself rather than to instances (objects) of the class. It allows you to access these members without creating an instance of the class.

1.Static Variables:

When a variable is declared as static, it is called a static variable or a class variable. The static variable is shared among all instances of the class. There is only one copy of the static variable that is shared across all objects of the class.

EXAMPLE:

```
public class MyClass {  
    // Static variable  
    public static int count = 0;  
  
    // Constructor  
    public MyClass() {  
        count++; // Incrementing static variable in the constructor  
    }  
  
    // Method to display count  
    public static void displayCount() {  
        System.out.println("Count: " + count);  
    }  
  
    // Main method  
    public static void main(String[] args) {  
  
        // Creating instances of MyClass  
        MyClass obj1 = new MyClass();  
        MyClass obj2 = new MyClass();  
        MyClass obj3 = new MyClass();  
  
        // Calling static method to display count  
        MyClass.displayCount();  
    }  
}
```

2.Static Methods:

When a method is declared as static, it is called a static method. Static methods belong to the class rather than individual objects. They can be invoked directly on the class itself, without the need to create an instance.

EXAMPLE:

```
public class MyClass {
    // Static method
    public static void staticMethod() {
        System.out.println("This is a static method");
    }
    // Main method
    public static void main(String[] args) {
        // Calling static method using class name
        MyClass.staticMethod();
    }
}
```

3.Static Block:

A static block is a block of code enclosed in curly braces {} that is executed when the class is loaded into memory. It is used to initialize static variables or perform any other static initialization tasks.

EXAMPLE:

```
public class MyClass {
    // Static variable
    public static int count;
    // Static block
    static {
        // Initializing static variable in the static block
        count = 0;
        System.out.println("Static block initialized");
    }
    // Main method
    public static void main(String[] args) {
        // Accessing the static variable
        System.out.println("Count: " + MyClass.count);    }    }
```

3.6 STRING CLASS AND ITS METHODS

The **String** class represents a sequence of characters. It is a built-in class and provides various methods to manipulate and work with **strings**.

1.Length:

The **length()** method returns the length (number of characters) of a string.

```
String str = "Hello, World!";
int length = str.length();
```

Output: length = 13

2.Concatenation:

The **concat()** method concatenates two strings and returns a new string.

```
String str1 = "Hello";
String str2 = "World";
String result = str1.concat(str2);
```

Output: result = "HelloWorld"

Alternatively, you can use the + operator for string concatenation.

```
String result = str1 + str2;
```

Output result = "HelloWorld"

3.Substring:

The **substring()** method returns a substring of the original string based on the specified starting and ending indexes.

```
String str = "Hello, World!";
String substr = str.substring(7, 12);
```

Output: substr = "World"

4.Comparison:

The **equals()** method checks if two strings have the same content.

```
String str1 = "Hello";
String str2 = "Hello";
boolean isEqual = str1.equals(str2);
```

Output: isEqual = true

The **equalsIgnoreCase()** method compares strings while ignoring case differences.

5. Conversion:

The **toUpperCase()** and **toLowerCase()** methods convert the string to uppercase and lowercase, respectively.

```
String str = "Hello, World!";
String uppercase = str.toUpperCase();
String lowercase = str.toLowerCase();
```

Output: uppercase = "HELLO, WORLD!"

lowercase = "hello, world!"

6. Searching:

The **indexOf()** method returns the index of the first occurrence of a specified substring within a string.

```
String str = "Hello, World!";
int index = str.indexOf("World");
```

Output: index = 7

7. Splitting:

The **split()** method splits a string into an array of substrings based on a specified delimiter.

```
String str = "Hello,World,Java";
String[] parts = str.split(",");
```

Output: parts = ["Hello", "World", "Java"]

8. Replacement:

The **replace()** method replaces all occurrences of a specified character or substring with another character or substring.

```
String str = "Hello, World!";
String replaced = str.replace("Hello", "Hi");
```

Output: replaced = "Hi, World!"

EXAMPLE:

```
public class StringOperationsExample {  
    public static void main(String[] args) {  
        // Length  
        String str = "Hello, world!";  
        int length = str.length();  
        System.out.println("Length of the string: " + length);  
  
        // Concatenation  
        String str1 = "Hello";  
        String str2 = "World";  
        String concatenatedString = str1.concat(" ").concat(str2);  
        System.out.println("Concatenated string: " + concatenatedString);  
  
        // Substring  
        String substring = str.substring(7);  
        System.out.println("Substring from index 7: " + substring);  
  
        // Comparison  
        String str3 = "hello";  
        boolean isEqualIgnoreCase = str1.equalsIgnoreCase(str3);  
        System.out.println("Comparison (ignoring case): " + isEqualIgnoreCase);  
  
        // Conversion  
        int number = 123;  
        String strNumber = String.valueOf(number);  
        System.out.println("Converted number to string: " + strNumber);  
  
        // Searching  
        boolean contains = str.contains("world");  
        System.out.println("Contains 'world': " + contains);  
    }  
}
```

```
// Splitting

String sentence = "This is a sentence";

String[] words = sentence.split("\\s+");

System.out.println("Splitting sentence into words:");

for (String word : words) {
    System.out.println(word);
}

// Replacement

String replacedString = str.replace("world", "Java");

System.out.println("Replaced 'world' with 'Java': " + replacedString);
}
}
```

Difference between String and String Buffer

No.	String	StringBuffer
1)	The String class is immutable.	The StringBuffer class is mutable.
2)	String is slow and consumes more memory when we concatenate too many strings because every time it creates new instance.	StringBuffer is fast and consumes less memory when we concatenate t strings.
3)	String class overrides the equals() method of Object class. So you can compare the contents of two strings by equals() method.	StringBuffer class doesn't override the equals() method of Object class.
4)	String class is slower while performing concatenation operation.	StringBuffer class is faster while performing concatenation operation.
5)	String class uses String constant pool.	StringBuffer uses Heap memory

3.7 I/O CLASSES AND FILE HANDLING

I/O (Input/Output) classes are used for reading input from various sources and writing output to different destinations. These classes provide functionality for file handling, reading from and writing to streams, and interacting with the user via the console.

1. File class:

The **File** class is used to represent files and directories. It provides methods to create, delete, rename, and query file properties.

```
import java.io.File;

// Creating a File object
File file = new File("path/to/file.txt");

// Checking if the file exists
boolean exists = file.exists();

// Creating a directory
File directory = new File("path/to/directory");
directory.mkdir();
```

2. Scanner class:

The Scanner class is used for reading user input from the console or parsing formatted data from different sources.

```
import java.util.Scanner;

// Reading user input from the console
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your name: ");
String name = scanner.nextLine();
System.out.println("Hello, " + name);

// Parsing data from a file
File file = new File("data.txt");
Scanner fileScanner = new Scanner(file);
while (fileScanner.hasNext()) {
    String data = fileScanner.nextLine();
    // Process the data
}
fileScanner.close();
```